Análisis de los algoritmos Monte Carlo Needles y Dartboard

Cálculo de pi con versiones seriales y paralelismo con OpenMP

Reto 2

Presentado por:
Samuel Herrera Marin
Jonathan Gaviria Ocampo
Tomás Marulanda Aristizabal

Presentado a:
Ramiro Andrés Barrios Valencia

HPC: High Performance Computing
Ingeniería de Sistemas y Computación
2026

Tabla de contenido

Resumen 3

1 Introducción 3

2 Objetivos 3

3 Marco conceptual y análisis de los algoritmos 4

4 Implementaciones seriales y verificación inicial 5

5 Diseño de la paralelización con OpenMP 6

6 Optimizaciones de CPU y memoria 8

7 Entorno y metodología experimental 9

8 Pruebas de corrección y resultados numéricos 10

9 Resultados del benchmark y comparación general 11

10 Perfilado y resultados de CPU y memoria 14

11 Análisis de las causas y límites de los resultados 16

12 Conclusiones 17

13 Bibliografía y fuentes 18

Anexo A Comandos y reproducción 18

Anexo B Evidencia de repeticiones y archivos 19

Resumen

Este informe presenta el análisis, la implementación y la evaluación de los algoritmos Monte Carlo Dartboard y Needles para estimar π. Partimos de dos programas seriales en C, verificamos sus fórmulas y ejecuciones, e incorporamos una implementación reproducible con OpenMP. Evaluamos el número de hilos, la planificación de las iteraciones, las opciones de compilación, el uso de SIMD y el costo de almacenar las muestras. La evaluación incluye 163 comprobaciones de corrección y 990 mediciones de desempeño, organizadas en 198 configuraciones con cinco repeticiones cada una.

Con diez millones de muestras, la menor mediana observada fue de 5,646 ms para Dartboard con compilación native y de 41,266 ms para Needles con O3, ambos con doce hilos y planificación dynamic. Sus speedups frente al serial del mismo binario fueron 7,06 y 10,18, respectivamente. El mayor speedup relativo de Dartboard fue 8,76 con O2, cuya base serial era más lenta. El uso de arreglos elevó el pico de memoria de aproximadamente 4,1 MiB a 156,7 MiB. El perfilado por fases mostró que la evaluación geométrica tiene un peso mucho mayor en Needles; el intento con gprof no produjo muestras utilizables en este entorno.

1 Introducción

Los métodos de Monte Carlo aproximan una cantidad mediante la frecuencia de eventos aleatorios relacionados con ella. En Dartboard se observa cuántos puntos caen dentro de un círculo; en Needles se observa cuántas agujas cruzan líneas paralelas. Ambas simulaciones permiten distribuir el trabajo porque cada muestra se evalúa de manera independiente. Esa propiedad convierte a estos algoritmos en un caso apropiado para estudiar paralelismo en memoria compartida.

El reto solicita analizar ambos algoritmos, implementar sus versiones seriales, estudiar e implementar opciones de paralelización y optimización de CPU y memoria, y explicar los resultados mediante evidencia experimental. El alcance de esta entrega corresponde a un equipo y tres integrantes. El benchmark mide las implementaciones de la aplicación; no se trasladan a este experimento las mediciones de matrices ni los benchmarks de otros equipos del documento usado como referencia de formato.

2 Objetivos

2.1 Objetivo general

Implementar y evaluar soluciones seriales y paralelas con OpenMP para estimar π mediante Needles y Dartboard, utilizando pruebas de corrección, benchmarks repetidos, perfilado y experimentos controlados de optimización de CPU y memoria.

2.2 Objetivos específicos

• Analizar las condiciones geométricas, los estimadores y la incertidumbre estadística de ambos algoritmos.

• Verificar la ejecución serial y la coherencia numérica de las implementaciones originales.

• Distribuir muestras con OpenMP y acumular aciertos con reducción, manteniendo reproducibilidad.

• Comparar static, dynamic, guided, SIMD y las compilaciones O2, O3 y native.

• Medir tiempo transcurrido, tiempo CPU y pico de memoria, y perfilar las fases de una variante instrumentada.

• Relacionar los resultados con el costo por muestra, la sincronización, el almacenamiento y las limitaciones del entorno.

3 Marco conceptual y análisis de los algoritmos

3.1 Monte Carlo y precisión estadística

En una simulación con N muestras, cada experimento aporta un indicador de éxito con valor cero o uno. Si la probabilidad de éxito es p, la proporción observada H/N aproxima p. Bajo el modelo ideal de muestras independientes, su varianza es p(1-p)/N. La incertidumbre típica disminuye como $1/\sqrt{N}$: multiplicar el tamaño de la simulación por cien reduce aproximadamente diez veces el error típico. El error observado de una ejecución concreta no tiene por qué disminuir monótonamente cuando aumenta N.

3.2 Dartboard

Se generan dos variables uniformes u y v en [0,1) y se transforman en coordenadas x=2u-1 e y=2v-1. El punto queda distribuido sobre el cuadrado de lado dos centrado en el origen. La condición $x^2+y^2\leq 1$ indica que pertenece al disco unitario. Como el área del disco es π y la del cuadrado es cuatro, la probabilidad ideal de acierto es π/4.

$$p = \pi/4 \qquad H = \text{número de puntos dentro del disco} \qquad \hat{\pi} = 4H/N$$

El algoritmo ejecuta dos generaciones pseudoaleatorias, operaciones aritméticas y una comparación por muestra. Su costo temporal es O(N) y su memoria auxiliar serial es O(1). El uso de distancias al cuadrado evita calcular una raíz cuadrada; esta decisión ya estaba presente en el código original.

3.3 Needles o aguja de Buffon

Se consideran líneas paralelas separadas una distancia D y agujas de longitud L, con $0<L\leq D$. Por simetría basta generar la distancia d del centro de la aguja a la línea más cercana en [0,D/2] y el ángulo θ en [0,π/2]. La aguja cruza una línea cuando la proyección perpendicular de su semilongitud alcanza esa distancia:

$$d \leq (L/2)\sin(\theta)$$

Al promediar la probabilidad condicional de cruce sobre los ángulos uniformes se obtiene $p=(2/\pi)\int_0^{\pi/2}(L/D)\sin(\theta)d\theta=2L/(\pi D)$. Si C es el número de cruces, el estimador es:

$$\hat{\pi} = 2LN/(DC), \text{ con } C>0; \text{ para } L=D=1 \text{ se reduce a } \hat{\pi} = 2N/C$$

La fórmula usada en este trabajo requiere $L\leq D$. Con agujas más largas se necesita otra expresión. Si no hay cruces, la estimación no está definida. Needles también usa tiempo O(N) y memoria auxiliar O(1), pero llama a sin en cada muestra. La formulación angular emplea una constante π conocida para generar θ; se mantiene como ejercicio de simulación y no como un procedimiento independiente para descubrir dígitos desconocidos de π.

3.4 Comparación de la incertidumbre

$$\sigma_{\text{Dartboard}} = \sqrt{[(4\pi-\pi^2)/N]} \approx 1{,}642/\sqrt{N}$$

$$\sigma_{\text{Needles}} \approx \sqrt{[\pi^2(\pi-2)/(2N)]} \approx 2{,}373/\sqrt{N}, \text{ para } L=D$$

La segunda expresión se obtiene por el método delta aplicado a 2/p. Dartboard es insesgado bajo el modelo ideal; Needles utiliza una transformación recíproca y presenta sesgo a tamaño finito cuando el estimador puede calcularse. Para igual incertidumbre asintótica, Needles necesita aproximadamente 2,09 veces las muestras de Dartboard en este caso. Más hilos con N fijo reducen tiempo, no incertidumbre. El error estándar estimado tampoco debe interpretarse con la aproximación normal cuando hay muy pocas muestras o proporciones iguales a cero o uno.

4 Implementaciones seriales y verificación inicial

4.1 Programas originales

Los archivos Needle.c y Dartboard.c se conservaron sin cambios. Ambos tienen un único bucle de simulación, no incluyen directivas OpenMP y no crean hilos ni tareas. Se recompilaron desde sus fuentes con GCC, sin -fopenmp, y no presentaron advertencias. Esta revisión confirma la ejecución serial del cálculo; no se basó únicamente en los ejecutables preexistentes.

Los originales fijan N=1 000 000 y usan srand(time(NULL)), por lo que las aproximaciones pueden cambiar entre ejecuciones. Su tiempo impreso se obtiene con clock(). Se conservan para documentar el punto de partida, mientras que el benchmark utiliza un reloj de tiempo transcurrido y una base serial reproducible.

4.2 Núcleo serial de Dartboard

```
for (long long i = 0; i < N; i++) {
    x = 2.0 * rand() / RAND_MAX - 1.0;
    y = 2.0 * rand() / RAND_MAX - 1.0;
    if (x * x + y * y <= 1.0) {
        dentro++;
    }
}
pi = 4.0 * (double)dentro / N;
```

La condición cuenta los puntos del disco y la expresión final transforma la frecuencia en π. No hay dependencia entre las condiciones geométricas de dos muestras; el estado del generador y la actualización del contador son los aspectos que requieren rediseño al pasar a paralelo.

4.3 Núcleo serial de Needles

```
for (long long i = 0; i < N; i++) {
    distancia = ((double)rand() / RAND_MAX) * (D / 2.0);
    angulo = ((double)rand() / RAND_MAX) * (PI / 2.0);
    if (distancia <= (L / 2.0) * sin(angulo)) {
        cruces++;
    }
}
if (cruces > 0) {
    pi = (2.0 * L * N) / (D * cruces);
}
```

El original usa L=D=1 y avisa si no puede calcular π por ausencia de cruces. La nueva implementación expresa ese caso como nan en el CSV. Ambas fórmulas son correctas bajo las condiciones descritas en el marco conceptual.

4.4 Resultados de la verificación de los originales

Algoritmo	N	Aciertos o cruces	π estimado	Tiempo (s)
Dartboard	1 000 000	785356	3,1414240000	0,024
Needles	1 000 000	636223	3,1435518678	0,063

Tabla 1. Ejecución serial original. Fuente: resultados/verificacion.json.

Los dos valores superaron el chequeo de coherencia de seis errores estándar respecto de π. Una ejecución no demuestra por sí sola la calidad del generador, pero la revisión de las fórmulas, la recompilación y este chequeo respaldan el punto de partida. Sus tiempos no se usan como denominador del speedup de OpenMP porque después se cambió el generador y el reloj.

5 Diseño de la paralelización con OpenMP

5.1 Muestreo reproducible sin estado compartido

La implementación unificada montecarlo.c asigna dos palabras pseudoaleatorias a cada índice i: 2i y 2i+1. La función uniform01 mezcla la semilla y la palabra mediante un finalizador SplitMix64. Sus operaciones uint64_t tienen desbordamiento modular definido y el resultado se transforma en un double de 53 bits en [0,1). El uso de mezclas SplitMix tiene como referente el trabajo de Steele, Lea y Flood [1]; aquí se usa indexación directa y no se implementa su interfaz completa de división de generadores.

```
uint64_t z = seed + UINT64_C(0x9e3779b97f4a7c15) * (word + 1);
z = (z ^ (z >> 30)) * UINT64_C(0xbf58476d1ce4e5b9);
z = (z ^ (z >> 27)) * UINT64_C(0x94d049bb133111eb);
z ^= z >> 31;
return (double)(z >> 11) * 0x1.0p-53;
```

Así, cualquier hilo puede reconstruir una muestra sin consultar el estado de otro. La semilla común es de solo lectura y no se necesita un arreglo de estados. Cambiar el número de hilos o el orden de ejecución conserva las muestras. No se atribuyen a OpenMP las diferencias frente al rand original: cada comparación usa el modo serial nuevo del mismo binario. Tampoco se afirma que las pruebas realizadas sustituyan una batería especializada de validación de números pseudoaleatorios.

5.2 Distribución y reducción del contador

```
uint64_t hits = 0;
#pragma omp parallel for default(none) shared(n, seed) \
        schedule(runtime) reduction(+:hits)
for (int64_t i = 0; i < n; ++i)
    hits += dart_hit(uniform01(seed, 2 * (uint64_t)i),
                     uniform01(seed, 2 * (uint64_t)i + 1));
```

parallel for distribuye las iteraciones entre los hilos del equipo. default(none) exige declarar el tratamiento de las variables que lo requieren. n y seed se comparten como lecturas; la variable de iteración y las variables locales de los predicados pertenecen a cada ejecución. reduction(+:hits)

crea acumulaciones privadas, inicializadas para la suma, y combina sus valores al finalizar [2]. Needles usa la misma estructura con needle_hit.

Por ejemplo, tres hilos podrían obtener 3, 2 y 4 aciertos en sus bloques; el resultado global sería 9. Una suma compartida sin protección introduciría una carrera. Proteger cada incremento con atomic o critical evitaría esa carrera, pero añadiría contención por muestra. La reducción concentra la combinación al final; el runtime puede implementar parte de esa combinación con una operación atómica. No se afirma que internamente no exista ninguna sincronización.

5.3 Configuración del equipo y del reparto

```
omp_set_dynamic(0);
omp_set_num_threads((int)threads);
omp_set_schedule(schedule,
                 schedule == omp_sched_static ? 0 : 1024);
```

Se desactiva el ajuste dinámico del tamaño del equipo y se solicita el número de hilos indicado por el usuario. Una región de diagnóstico consulta omp_get_num_threads antes del intervalo cronometrado. Esto inicializa el equipo y registra el número real de participantes. Los modos serial y buffered no entran en esa región ni en otras regiones paralelas.

Planificación	Reparto implementado	Análisis
static	Bloques contiguos; tamaño por defecto del runtime.	Poca gestión; razonable si los hilos progresan de manera parecida.
dynamic	Bloques de 1024 muestras solicitados al terminar cada bloque.	Añade gestión, pero permite redistribuir trabajo pendiente.
guided	Bloques decrecientes, con mínimo de 1024.	Busca equilibrar disponibilidad y costo de asignación.

Tabla 2. Alternativas de planificación implementadas con schedule(runtime) [3].

omp_set_dynamic(0) controla el tamaño del equipo; schedule(dynamic) controla el reparto de iteraciones. Son decisiones distintas. No se usó collapse porque el trabajo principal es un único bucle de muestras. Tampoco se implementaron tareas: para este trabajo regular el for proporciona una división directa sin crear una tarea por muestra.

5.4 Variante con SIMD

```
#pragma omp parallel for simd default(none) shared(n, seed) \
        schedule(runtime) reduction(+:hits)
```

El modo simd añade la posibilidad de ejecutar varias iteraciones mediante instrucciones vectoriales dentro del trabajo de cada hilo [4]. La directiva no garantiza una mejora efectiva: depende del cuerpo, la arquitectura y las decisiones del compilador. Se conservó como alternativa evaluada y se contrastó tanto con tiempos como con diagnósticos de GCC.

6 Optimizaciones de CPU y memoria

6.1 Opciones de compilación y simplificación del cálculo

Binario	Opciones específicas	Propósito
O2	-O2	Base optimizada general.
O3	-O3	Transformaciones adicionales de optimización.
native	-O3 -march=native	Especialización para la CPU local.
profile	-O2 -g -pg -fno-inline	Intento con gprof; excluido del benchmark.

Tabla 3. Variantes construidas por build.py. Comandos completos en resultados/compilacion.txt.

Las variantes nuevas comparten -fopenmp, -ffp-contract=off y -fno-math-errno. La contracción desactivada evita diferencias por operaciones fusionadas como FMA; no consultar errno después de sin permite habilitar la segunda opción. No se activa -ffast-math. Según GCC, el nivel de optimización y las opciones de arquitectura cambian las transformaciones permitidas [5]. Un binario native debe recompilarse al trasladarlo a una CPU incompatible.

En Needles se cancelaron los factores 1/2 de la condición para L=D=1. En Dartboard se mantuvo la comparación de distancias al cuadrado. Los predicados devuelven un booleano que se suma al contador; las funciones pequeñas static inline y las constantes visibles facilitan decisiones del compilador. No se midió aisladamente cada transformación, por lo que no se asigna un porcentaje de mejora individual a estas simplificaciones.

6.2 Generación y consumo inmediato de las muestras

Los modos serial, omp y simd generan una muestra, evalúan su condición y la descartan. No almacenan puntos ni agujas y no reservan memoria dentro del bucle. La memoria del algoritmo es O(1) en serial y O(p) para p hilos, por el trabajo privado y los contadores, además del runtime. Se evita una escritura compartida por muestra. No se añadió padding manual ni se ejecutó un experimento específico de false sharing.

6.3 Contraste con dos arreglos de muestras

buffered es una variante experimental serial: reserva un bloque contiguo para dos arreglos, escribe los valores u y v de todas las muestras y luego los recorre para evaluarlas. Los predicados y las muestras son los mismos que en la versión sin arreglos. Su memoria adicional es 2N×sizeof(double)=16N bytes. Con N=10 millones equivale a 152,59 MiB.

```
double *uv = malloc((size_t)n * 2 * sizeof(double));
double *v = uv + n;
/* Primera fase: generar y guardar uv[i] y v[i]. */
/* Segunda fase: leer ambos arreglos y contar aciertos. */
free(uv);
```

Esta variante permite estudiar el costo de reservar, escribir y leer muestras, y separar fases para el perfilado. Los programas originales ya empleaban memoria constante; por ello la fusión conserva una buena decisión original y no se presenta como la eliminación de un problema que existiera en esos archivos.

7 Entorno y metodología experimental

7.1 Entorno registrado

Componente	Valor
Procesador	Intel(R) Core(TM) i5-10400 CPU @ 2.90GHz
Procesadores lógicos	12
Sistema operativo	Windows-10-10.0.19045-SP0
Compilador	gcc (Rev5, Built by MSYS2 project) 16.1.0
Python	3.11.1
Fecha de la campaña	4 de noviembre de 2026
Variables del benchmark	OMP_DYNAMIC=FALSE; OMP_WAIT_POLICY=PASSIVE
Semillas	Muestras: 42; orden aleatorio de ejecución: 2026

Tabla 4. Fuente: resultados/entorno.json. La frecuencia del nombre de CPU es nominal, no una frecuencia medida.

No se fijó afinidad ni se controlaron temperatura, frecuencia efectiva, plan de energía o carga externa. No se extrapolan estos resultados a otros equipos. Los doce participantes son procesadores lógicos y no doce núcleos con todos los recursos independientes.

7.2 Diseño de la campaña

Se combinaron tres compilaciones, dos algoritmos y tres tamaños: 100 000, 1 000 000 y 10 000 000 muestras. Para cada combinación se ejecutaron once configuraciones: serial; buffered; OpenMP static con 1, 2, 4, 6 y 12 hilos; SIMD static con 1 y 12 hilos; y OpenMP con 12 hilos usando dynamic y guided. Cada configuración se repitió cinco veces.

$$3 \times 2 \times 3 \times 11 \times 5 = 990 \text{ mediciones válidas}$$

benchmark.py mezcla el orden de las ejecuciones con una semilla fija y ejecuta los procesos uno por uno. Realiza un calentamiento por binario y algoritmo, además de la inicialización del equipo dentro de cada proceso paralelo. La semilla 42 conserva el trabajo entre repeticiones: estas repeticiones miden variación temporal y no nuevos experimentos de π. El perfilado y la campaña definitiva se ejecutaron por separado.

7.3 Métricas e intervalo medido

wall_s mide generación, evaluación y reducción después de inicializar el equipo OpenMP. No incluye el arranque del proceso, la estimación final de π ni su impresión. En buffered incluye reserva y liberación. process_wall_s se mide desde Python e incluye la ejecución completa del proceso. Se reportan mediana, mínimo, máximo y desviación estándar temporal.

$$S(p) = \text{mediana}(T_{serial})/\text{mediana}(T_p) \qquad E(p) = S(p)/p$$

El denominador serial pertenece al mismo binario, algoritmo, N y semilla. La mediana reduce la sensibilidad a una ejecución extrema, pero cinco repeticiones no eliminan la variabilidad. La

eficiencia divide el speedup entre el número real de hilos. Elegir la mejor mediana entre muchas alternativas también introduce sesgo de selección.

En la exploración inicial, omp_get_wtime devolvió algunos intervalos cero en este entorno. La campaña se descartó y se repitió utilizando QueryPerformanceCounter en Windows, recomendado para intervalos de alta resolución [6]. Fuera de Windows se usa omp_get_wtime. Este cambio mejora la medición, no acelera el algoritmo.

cpu_s proviene del tiempo CPU acumulado del proceso; en Windows se obtiene con GetProcessTimes. Su resolución resulta insuficiente para algunas ejecuciones cortas: un cero no indica ausencia de trabajo. peak_rss_kib registra el pico residente de todo el proceso, incluidas bibliotecas y runtime. buffer_bytes identifica únicamente el espacio de los arreglos. No se midieron fallos de caché ni ancho de banda hardware.

8 Pruebas de corrección y resultados numéricos

8.1 Comprobaciones automatizadas

Grupo de pruebas	Casos	Criterio y resultado
Referencia Python	108	Igualdad exacta de conteos con C en tres compilaciones, dos algoritmos, tres casos y seis modos/equipos.
Planificación	36	N=100003, cuatro hilos; static, dynamic y guided en omp y simd. Igualdad con serial.
Entradas inválidas	7	Rechazo con código 2 de N inválido, hilos inválidos, algoritmo inexistente y desbordamientos.
Coherencia estadística	10	Cinco semillas por algoritmo, N=1000000. Estimación finita y error menor que seis errores estándar.
Originales	2	Recompilación y chequeo estadístico de ambos programas.
Total	163	Todas las comprobaciones fueron satisfactorias.

Tabla 5. Pruebas implementadas en verify.py y resultado en resultados/verificacion.json.

Los casos de la referencia fueron (N=1, semilla=0), (N=17, semilla=42) y (N=10003, semilla=18446744073709551615). Se compararon serial, buffered, omp con 1, 2 y 4 hilos, y simd con 4 hilos. La referencia Python expresa la geometría de forma separada y reproduce el mezclado entero de 64 bits. No es una biblioteca RNG independiente. Los tamaños no divisibles entre hilos permiten revisar el reparto de iteraciones sobrantes.

Los siete errores probados fueron N=0, N negativo, cero hilos, algoritmo inexistente, más de un hilo en serial, semilla superior al máximo de uint64_t y N superior al máximo de int64_t. El chequeo estadístico usa una tolerancia amplia de seis errores estándar, no un número fijo de decimales. Además, durante las 990 mediciones se comprobó la igualdad de conteos entre configuraciones para cada algoritmo y N.

8.2 Variación entre semillas

Semilla	Dartboard π estimado	Needles π estimado
1	3,1407600000	3,1414730996
7	3,1424520000	3,1441152381
42	3,1419280000	3,1398554411
2026	3,1407760000	3,1444365484
987654321	3,1408560000	3,1419912998

Tabla 6. Un millón de muestras por semilla, modo serial native. Todos los chequeos de seis errores estándar pasaron.

La variación es propia del método. Para la semilla 42 el error estándar estimado fue aproximadamente 0,001642 en Dartboard y 0,002370 en Needles. Que una semilla resulte más cercana a π no implica que ese modo tenga sistemáticamente mayor precisión.

8.3 Coincidencia exacta con diez millones de muestras

Algoritmo	Aciertos o cruces	π estimado	Error absoluto	Error estándar
Dartboard	7855863	3,142345200000	0,0007525464	0,0005191385
Needles	6368382	3,140515126134	0,0010775275	0,0007499571

Tabla 7. Semilla 42. Los conteos coinciden entre modos, hilos y compilaciones.

9 Resultados del benchmark y comparación general

9.1 Línea base serial y compilación

Compilación	Dartboard (ms)	Needles (ms)
O2	50,623	433,635
O3	37,679	420,053
native	39,844	419,194

Tabla 8. Medianas seriales, N=10000000, cinco repeticiones por configuración.

O3 redujo la mediana respecto de O2 aproximadamente 1,34 veces en Dartboard y 1,03 veces en Needles. native no mejoró de forma consistente sobre O3. Los cambios pequeños deben interpretarse junto con la dispersión y no como una ganancia garantizada del flag. Needles sigue siendo más costoso por muestra, de acuerdo con la evaluación trigonométrica que realiza en cada iteración.

9.2 Escalabilidad de Dartboard

Modo	Hilos	Reparto	Tiempo (ms)	Speedup	Eficiencia
serial	1	static	39,844	1,00	-
omp	1	static	33,024	1,21	120,7 %
omp	2	static	17,365	2,29	114,7 %
omp	4	static	12,571	3,17	79,2 %
omp	6	static	9,989	3,99	66,5 %
omp	12	static	7,805	5,11	42,5 %
omp	12	dynamic	5,646	7,06	58,8 %
omp	12	guided	5,837	6,83	56,9 %
simd	12	static	8,402	4,74	39,5 %

Tabla 9. Dartboard, N=10000000, compilación native. Base: serial del mismo binario.

El paso de 6 a 12 hilos con static aporta una mejora menor que duplicar el rendimiento: la mediana pasa de 9,989 a 7,805 ms. dynamic con 12 hilos obtuvo 5,646 ms, la menor mediana observada para Dartboard entre todas las compilaciones. La eficiencia de esta configuración fue 58,8 % respecto de su base serial native.

9.3 Escalabilidad de Needles

Modo	Hilos	Reparto	Tiempo (ms)	Speedup	Eficiencia
serial	1	static	419,194	1,00	-
omp	1	static	412,795	1,02	101,5 %
omp	2	static	207,579	2,02	101,0 %
omp	4	static	110,113	3,81	95,2 %
omp	6	static	77,737	5,39	89,9 %
omp	12	static	67,459	6,21	51,8 %
omp	12	dynamic	44,258	9,47	78,9 %
omp	12	guided	46,018	9,11	75,9 %
simd	12	static	63,671	6,58	54,9 %

Tabla 10. Needles, N=10000000, compilación native. Base: serial del mismo binario.

Needles obtuvo 9,47 de speedup con native, doce hilos y dynamic. Su mayor cantidad de trabajo por muestra permite amortizar mejor parte del costo de distribución y sincronización. Aun así, Dartboard mantiene menor tiempo absoluto. El mayor speedup no significa que Needles sea la opción más rápida para estimar π.

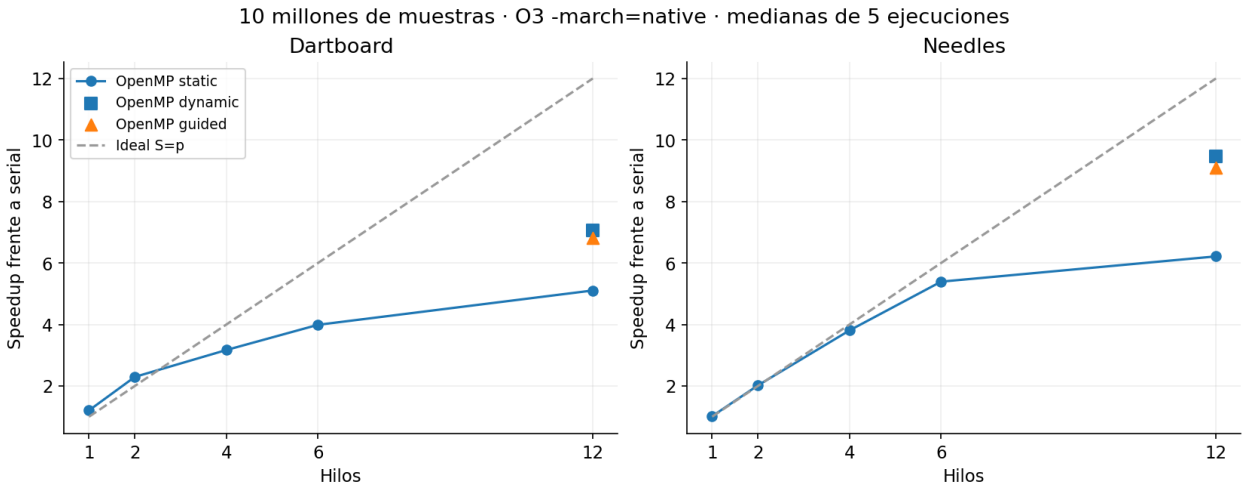


Figura 1. Speedup con N=10000000 y native. Las curvas usan medianas de cinco ejecuciones; la línea discontinua indica S=p. Fuente: elaboración propia desde resumen.csv.

9.4 Menor tiempo frente a mayor speedup relativo

Algoritmo	Binario	Serial (ms)	OpenMP (ms)	Speedup
Dartboard	O2	50,623	5,778	8,76
Dartboard	native	39,844	5,646	7,06
Needles	O3	420,053	41,266	10,18
Needles	native	419,194	44,258	9,47

Tabla 11. Doce hilos y dynamic, N=10000000. Cada speedup emplea su propio serial.

Dartboard O2 tuvo mayor speedup relativo, 8,76, porque parte de una base serial más lenta; native tuvo menor tiempo paralelo, 5,646 ms, y speedup 7,06. Para Needles, la menor mediana fue 41,266 ms con O3, equivalente a 10,18 de speedup y 84,8 % de eficiencia. Las medianas más cercanas no deben interpretarse como una diferencia estadísticamente demostrada entre compilaciones.

9.5 Tamaño de la simulación y costo completo

Algoritmo	N	Serial (ms)	OMP dynamic (ms)	Speedup
Dartboard	100000	0,327	0,094	3,49
Dartboard	1000000	3,352	0,646	5,19
Dartboard	10000000	39,844	5,646	7,06
Needles	100000	4,101	0,459	8,94
Needles	1000000	41,272	4,374	9,44
Needles	10000000	419,194	44,258	9,47

Tabla 12. Comparación por tamaño, compilación native y doce hilos en OpenMP.

El speedup anterior corresponde al cálculo. Con N=100000, Dartboard serial native necesitó 0,327 ms de cálculo y 10,550 ms como proceso completo. OpenMP con doce hilos static redujo el

cálculo a 0,137 ms, pero el proceso completo tardó 12,387 ms. En ese caso, el arranque absorbe la mejora para una ejecución aislada. Reutilizar un equipo de hilos en una aplicación persistente podría modificar esa relación.

9.6 Dispersión y planificación

Algoritmo	Reparto	Mediana (ms)	Mínimo (ms)	Máximo (ms)	Desv. (ms)
Dartboard	static	7,805	5,662	8,095	0,999
Dartboard	dynamic	5,646	5,180	6,160	0,357
Dartboard	guided	5,837	5,134	9,105	1,578
Needles	static	67,459	44,065	68,801	10,734
Needles	dynamic	44,258	39,999	44,676	1,936
Needles	guided	46,018	41,642	53,102	4,665

Tabla 13. Cinco repeticiones, native, N=10000000 y doce hilos.

En esta campaña dynamic superó a static y presentó menor dispersión para las configuraciones comparadas. Es compatible con una mejor compensación de retrasos entre hilos, aunque las iteraciones tengan un costo parecido. Sin afinidad y carga controladas no se puede atribuir toda la diferencia al algoritmo de reparto ni afirmar que dynamic siempre será mejor.

10 Perfilado y resultados de CPU y memoria

10.1 Diagnósticos de vectorización y comparación SIMD

Los archivos vector_O2.txt, vector_O3.txt y vector_native.txt registran decisiones de vectorización de GCC. Para los bucles de muestras se observan mensajes como couldn't vectorize loop y SLP build failed. Algunas notas positivas se refieren a partes auxiliares de las regiones de reducción, de modo que no prueban que todo el cuerpo Monte Carlo esté vectorizado. El desensamblado del binario native mostró llamadas escalares a sin; no se observó aritmética de doble precisión empaquetada en la inspección realizada.

Algoritmo	OMP static 12 (ms)	SIMD static 12 (ms)	Interpretación
Dartboard	7,805	8,402	No hubo mejora de la mediana.
Needles	67,459	63,671	Diferencia pequeña frente a la dispersión.

Tabla 14. N=10000000, native; comparación con el mismo número de hilos y reparto.

Las mezclas enteras de 64 bits del RNG y la evaluación de sin son obstáculos posibles para una vectorización eficiente. Los registros prueban el resultado del compilador, pero no aíslan cuánto cuesta cada obstáculo. La conclusión respaldada es que SIMD no mostró una mejora consistente en esta implementación.

10.2 Pico de memoria y costo de almacenar muestras

Algoritmo	Modo	Tiempo (ms)	Pico RSS (MiB)	Arreglos (bytes)
Dartboard	serial	39,844	4,12	0
Dartboard	buffered	73,709	156,69	160000000
Needles	serial	419,194	4,12	0
Needles	buffered	443,336	156,69	160000000

Tabla 15. Medianas de cinco ejecuciones, N=10000000, compilación native.

El pico RSS aumentó de unos 4,1 MiB a 156,7 MiB al guardar las muestras. La diferencia es coherente con los 152,59 MiB de los arreglos. Generar y consumir inmediatamente fue cerca de 1,85 veces más rápido que buffered en Dartboard y 1,06 veces en Needles. Reserva, escritura, lectura y evaluación contribuyen al tiempo; no se midieron contadores para atribuir la diferencia solamente a caché.

10.3 Intento con gprof y alternativa instrumentada

profile.py ejecutó la compilación -pg sobre treinta millones de muestras en serial y procesó los archivos gmon.out con gprof. En este runtime, ambos perfiles quedaron sin muestras o funciones utilizables. Se conservaron los archivos como evidencia del intento, pero no se reportan porcentajes válidos de gprof.

Como alternativa, se midieron generación más escritura y lectura más evaluación de buffered, usando native sin -pg. Se descartó un calentamiento por algoritmo y se conservaron cinco ejecuciones con treinta millones de muestras. Este es perfilado instrumentado de esa variante, distinto de un perfil de muestreo del ejecutable fusionado.

Algoritmo	Generación (ms)	Evaluación (ms)	Total (ms)	Evaluación / fases
Dartboard	145,288	40,907	218,789	22,0 %
Needles	139,308	1123,755	1294,086	89,0 %

Tabla 16. Fuente: perfil_fases.csv y perfil_estado.json. Generación incluye escritura; evaluación incluye lectura.

En Dartboard la generación y escritura domina estas dos fases; en Needles la evaluación representa aproximadamente el 89 % de su suma. No significa que sin consuma por sí solo el 89 %: la fase también contiene lecturas, aritmética y conteo. Las medianas de fases no tienen por qué sumar la mediana total, que además incluye reserva, liberación y otras operaciones. Tampoco se transfieren estos porcentajes directamente al bucle sin arreglos.

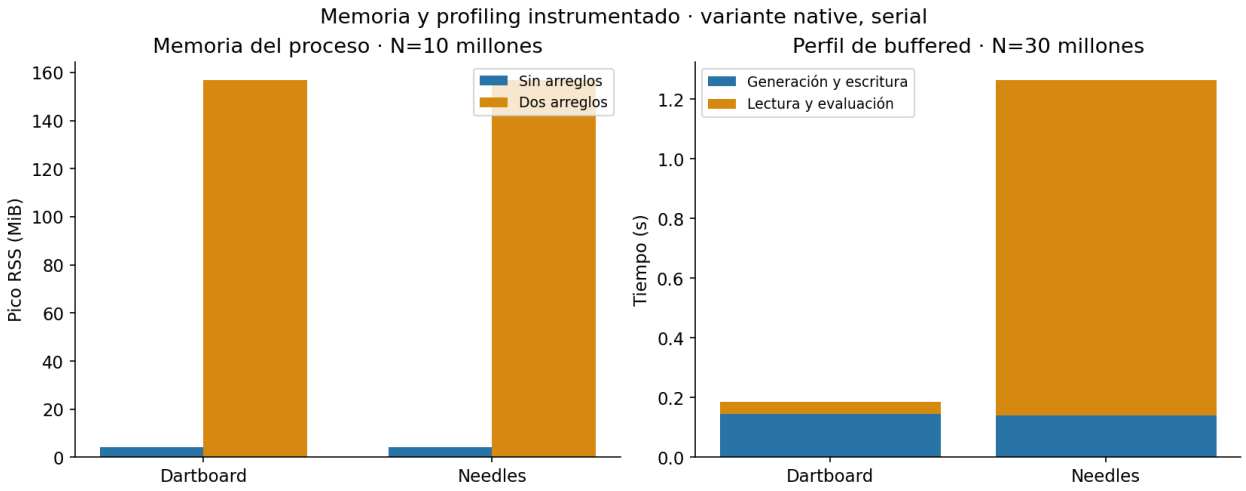


Figura 2. Memoria con N=10000000 y perfil instrumentado con N=30000000. Compilación native, ejecución serial. Fuente: elaboración propia a partir de las mediciones.

11 Análisis de las causas y límites de los resultados

11.1 Trabajo por muestra y escalabilidad

Needles evalúa una función trigonométrica por muestra, mientras Dartboard utiliza principalmente operaciones aritméticas simples. Esa diferencia está reflejada en los tiempos seriales y en el perfil por fases. Un mayor trabajo local puede amortizar la distribución y las barreras de OpenMP, lo que es compatible con el mayor speedup relativo de Needles. Sin embargo, para un mismo N, Dartboard sigue tardando menos y tiene menor varianza ideal.

Duplicar hilos no duplica necesariamente el rendimiento. La creación o activación del equipo, la asignación de bloques, la reducción y las esperas añaden costo. Los procesadores lógicos pueden compartir recursos de ejecución y la disponibilidad del sistema puede variar. Estas son explicaciones plausibles; la campaña no cuantificó por separado instrucciones, contención y tiempo de espera con contadores hardware.

11.2 Eficiencias superiores al cien por ciento

Con native, Dartboard obtiene un speedup de 1,21 incluso con OpenMP de un hilo, y de 2,29 con dos hilos. Esto no demuestra una capacidad física superlineal. El compilador genera rutas diferentes para el bucle serial y la región OpenMP, y las mediciones presentan variación. Usando el OpenMP de un hilo como referencia secundaria, la escalabilidad static a doce hilos es aproximadamente 4,23, frente a 5,11 con el serial como base.

11.3 Planificación y variabilidad del entorno

La mediana native de Needles static con doce hilos fue 67,459 ms, con extremos de 44,065 y 68,801 ms. dynamic obtuvo 44,258 ms, con extremos de 39,999 y 44,676 ms. Redistribuir bloques puede compensar hilos retrasados, pero no se demostró un desequilibrio intrínseco importante entre muestras. Para aislar el efecto sería necesario fijar afinidad, controlar energía y carga externa, y ampliar las repeticiones.

11.4 Alcance de la evidencia

Las mediciones demuestran igualdad de conteos en los casos evaluados, reducción de tiempos con OpenMP y el costo real de guardar muestras. No demuestran calidad universal del RNG, superioridad de un schedule para todo equipo ni una vectorización completa del cuerpo. El perfil instrumentado aporta tiempos de fases; gprof no aportó datos útiles. El benchmark se realizó en una sola máquina y no se generaliza a los otros computadores mencionados en el informe de referencia.

La mejora de memoria conserva el diseño O(1) de los originales. No se implementaron ni validaron afinidad, padding, bibliotecas de seno vectorial o generadores alternativos; son posibles líneas posteriores, no resultados de esta entrega. La campaña usa tiempos del cálculo calentado, acompañados de tiempos completos para reconocer cuándo el arranque cambia la conclusión.

12 Conclusiones

• Las versiones originales son seriales y sus fórmulas son correctas para Dartboard y para Needles con L=D=1. La recompilación no presentó advertencias y sus ejecuciones superaron el chequeo estadístico.

• El muestreo por índice y la reducción entera permitieron paralelizar ambos algoritmos conservando exactamente los conteos. Las 163 comprobaciones y los controles de las 990 mediciones respaldan la corrección en los casos evaluados.

• OpenMP produjo mejoras importantes cuando hubo trabajo suficiente. Para diez millones de muestras, las menores medianas fueron 5,646 ms en Dartboard native y 41,266 ms en Needles O3, con doce hilos y dynamic.

• El menor tiempo observado y el mayor speedup no siempre corresponden al mismo binario. Dartboard native obtuvo 7,06 de speedup; O2 obtuvo 8,76 frente a un serial más lento. Needles O3 alcanzó 10,18.

• O3 fue útil especialmente en Dartboard. native y SIMD no produjeron mejoras consistentes adicionales, por lo que deben evaluarse con mediciones y diagnósticos, no asumirse superiores.

• Generar y consumir muestras inmediatamente mantuvo el pico residente cerca de 4,1 MiB. Al guardar dos arreglos aumentó a unos 156,7 MiB y el tiempo se incrementó, especialmente en Dartboard.

• El perfilado por fases encontró mayor peso de la evaluación en Needles, coherente con el uso de sin. El intento con gprof no produjo muestras útiles y no se presentan porcentajes de esa herramienta.

• Dartboard fue más rápido y tiene menor varianza ideal por muestra en las condiciones estudiadas. Para ampliar la validez de las conclusiones conviene repetir en otros equipos, controlar afinidad y energía, y usar contadores y un profiler de muestreo operativo.

13 Bibliografía y fuentes

[1] Steele, G. L., Lea, D. y Flood, C. H. (2014). Fast Splittable Pseudorandom Number Generators. OOPSLA. DOI 10.1145/2660193.2660195.
https://gee.cs.oswego.edu/dl/papers/oopsla14.pdf

[2] OpenMP Architecture Review Board. (2021). OpenMP API Specification 5.2. Reduction clause.
https://www.openmp.org/spec-html/5.2/openmpsu52.html

[3] OpenMP Architecture Review Board. OpenMP API 5.2 Reference Guide. Planificación y funciones del runtime.
https://www.openmp.org/wp-content/uploads/OpenMPRefGuide-5.2-Web-2024.pdf

[4] OpenMP Architecture Review Board. (2021). OpenMP API Specification 5.2. SIMD construct.
https://www.openmp.org/spec-html/5.2/openmpse60.html

[5] GNU Project. Using the GNU Compiler Collection. Optimize Options y Developer Options.
https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

[6] Microsoft. Acquiring high-resolution time stamps. Documentación de Windows.
https://learn.microsoft.com/en-us/windows/win32/sysinfo/acquiring-high-resolution-time-stamps

[7] Fuentes y datos experimentales del proyecto (2026). Needle.c, Dartboard.c, montecarlo.c, build.py, verify.py, benchmark.py y profile.py; mediciones.csv, resumen.csv, verificacion.json, entorno.json, perfil_fases.csv, perfil_estado.json y vector_*.txt. Carpeta del proyecto: Reto 2. Campaña del 4 de noviembre de 2026.

Las fuentes en línea se consultaron el 4 de noviembre de 2026. Los resultados numéricos del informe se tomaron de los registros del proyecto [7], no de estimaciones de desempeño publicadas por terceros.

Anexo A Comandos y reproducción

Desde PowerShell, ubicados en la carpeta del proyecto, la secuencia utilizada es la siguiente. El perfilado y el benchmark deben ejecutarse por separado. Los scripts de compilación, prueba y benchmark requieren Python estándar y GCC con OpenMP; las figuras usan matplotlib como dependencia opcional.

```
cd "E:\HPC\Reto 2"
python build.py
python verify.py
python profile.py
python benchmark.py --repeats 5
python graficas.py
```

Ejemplos de ejecución directa con el mismo algoritmo, tamaño y semilla:

```
.\build\mc_native.exe dartboard serial 10000000 1 42 static
.\build\mc_native.exe dartboard omp 10000000 12 42 dynamic
.\build\mc_O3.exe needles serial 10000000 1 42 static
.\build\mc_O3.exe needles omp 10000000 12 42 dynamic
.\build\mc_native.exe dartboard buffered 10000000 1 42 static
```

Ejemplo de compilación manual en MinGW-w64, con continuación de líneas de PowerShell:

```
gcc -std=c11 -O3 -march=native -Wall -Wextra -Wpedantic `
    -fopenmp -ffp-contract=off -fno-math-errno `
    montecarlo.c -o mc_native.exe -lm -lpsapi
```

El archivo resultados/compilacion.txt conserva los comandos completos realmente utilizados, incluidos -Werror y los reportes de vectorización. En Linux no se añade -lpsapi y los ejecutables no llevan la extensión .exe. El código valida N positivo, número de hilos válido y rangos enteros; serial y buffered exigen un hilo.

Anexo B Evidencia de repeticiones y archivos

Las siguientes tablas muestran las cinco ejecuciones de las configuraciones con menor mediana de tiempo para cada algoritmo con diez millones de muestras. Las 990 filas completas permanecen en mediciones.csv. No se seleccionaron repeticiones individuales para calcular los tiempos publicados.

Repetición	Cálculo (ms)	Aciertos o cruces	Proceso completo (ms)
1	5,6065	7855863	16,9555
2	5,6458	7855863	15,5800
3	5,1804	7855863	15,4716
4	5,8367	7855863	18,8652
5	6,1600	7855863	18,8204
Mediana	5,646	Mismo conteo	16,9555

Dartboard native. Doce hilos, dynamic, N=10000000. Fuente: mediciones.csv.

Repetición	Cálculo (ms)	Aciertos o cruces	Proceso completo (ms)
1	40,3426	6368382	50,4879
2	41,2657	6368382	53,2515
3	44,6454	6368382	55,0839
4	40,7567	6368382	52,3297
5	45,3694	6368382	55,0665
Mediana	41,266	Mismo conteo	53,2515

Needles O3. Doce hilos, dynamic, N=10000000. Fuente: mediciones.csv.

Las fuentes y archivos identificados en [7] conservan las implementaciones, los chequeos, la campaña y el entorno. perfil_fases.csv y perfil_estado.json documentan la instrumentación; las carpetas perfil_dartboard y perfil_needles conservan el intento con gprof. Los archivos vector_*.txt registran las decisiones SIMD del compilador.